

Mastering Spring Boot

A Comprehensive Guide from Zero to Production

The Complete Beginner's Tutorial

June 2, 2026

Contents

1	Introduction to Spring Boot	3
1.1	What is the Spring Framework?	3
1.2	The Challenge: Configuration Overhead	3
1.3	Enter Spring Boot	3
1.3.1	Key Features of Spring Boot	3
2	Setting Up Your Environment	5
2.1	Prerequisites	5
2.2	Spring Initializr	5
3	Anatomy of a Spring Boot Project	6
3.1	The File Structure	6
3.2	The Maven POM File	6
3.3	The Application Class	7
4	Dependency Injection and Beans	8
4.1	Inversion of Control (IoC)	8
4.2	Beans and Stereotype Annotations	8
4.3	Dependency Injection	8
5	Building a RESTful API	10
5.1	The RestController	10
5.2	HTTP Methods and Mappings	10
5.3	Handling Input Parameters	10
5.3.1	Path Variables	10
5.3.2	Request Parameters	10
5.3.3	Request Body	10
6	Data Access with Spring Data JPA	12
6.1	Setting Up Dependencies	12
6.2	Configuring the Database	12
6.3	Creating an Entity	12
6.4	Creating a Repository	13
7	Architectural Best Practices	15
7.1	The Layered Architecture	15
7.2	Data Transfer Objects (DTOs)	15
8	Exception Handling	16
8.1	Custom Exceptions	16
8.2	Global Exception Handler (@ControllerAdvice)	16
9	Application Configuration and Profiles	18
9.1	The @Value Annotation	18
9.2	Spring Profiles	18

10 Testing Spring Boot Applications	19
10.1 Unit Testing	19
11 Production-Ready Features	21
11.1 Adding Actuator	21
12 Packaging and Deployment	22
12.1 The Executable JAR	22
12.2 Running the Application	22
12.3 Next Steps in Your Journey	22

Chapter 1

Introduction to Spring Boot

1.1 What is the Spring Framework?

Java is one of the world's most widely used programming languages, recognized for its flexibility, platform independence, and robustness in enterprise application development. However, as enterprise applications grew more sophisticated, managing the complexity of Java projects became a daunting task. Developers had to manage heavy configuration files, manual object lifecycles, and complex database connections.

Introduced in 2003, the Spring Framework solved many of these issues by giving developers a cleaner, more modular way to build Java applications. At its core, Spring is a Dependency Injection (DI) and Inversion of Control (IoC) container that manages the life-cycle of your application's objects.

1.2 The Challenge: Configuration Overhead

While the Spring Framework was a massive improvement over traditional Java EE, it brought its own challenges. Setting up a Spring application typically required writing massive amounts of XML configuration or boilerplate Java code. Integrating third-party libraries (like Hibernate for databases, or Tomcat as a web server) required manually configuring compatibility and versions.

1.3 Enter Spring Boot

Spring Boot is an extension of the Spring Framework designed to simplify the setup and development of Spring applications. It takes an "opinionated" view of the Spring platform and third-party libraries, allowing you to get started with minimum fuss.

1.3.1 Key Features of Spring Boot

- **Auto-Configuration:** Spring Boot looks at the dependencies you have added to your project and automatically configures the necessary beans and settings. If you add a database driver, Spring Boot automatically configures the database connection.
- **Starter Dependencies:** Spring Boot provides pre-configured dependency descriptors (Starters) that group related libraries together. For example, `spring-boot-starter-web` includes everything you need to build a RESTful web service (Spring MVC, Jackson for JSON, and Tomcat).
- **Embedded Servers:** Traditionally, Java web applications were packaged as WAR files and deployed into an external server like Apache Tomcat. Spring Boot embeds Tom-

cat (or Jetty, or Undertow) directly inside your application. You simply run your application as a standard Java program (executable JAR).

- **Production-Ready Features:** Spring Boot Actuator provides built-in endpoints for monitoring application health, metrics, and environment properties without writing any additional code.

Chapter 2

Setting Up Your Environment

Before writing any code, you need to set up your development environment. Spring Boot makes this process incredibly streamlined.

2.1 Prerequisites

- **Java Development Kit (JDK):** You need a modern JDK installed on your machine. Spring Boot 3 requires JDK 17 as a minimum, but JDK 21 is highly recommended.
- **Integrated Development Environment (IDE):** Popular choices include IntelliJ IDEA (Community or Ultimate), Eclipse (with Spring Tools), or Visual Studio Code with the Java Extension Pack.
- **Build Tool:** Maven or Gradle. These tools manage your project's dependencies and build lifecycle. We will use Maven in this guide.

2.2 Spring Initializr

The absolute best way to start a new Spring Boot project is by using the Spring Initializr. It is a web-based tool provided by the Spring team.

1. Open your web browser and navigate to <https://start.spring.io>.
2. **Project:** Choose Maven.
3. **Language:** Choose Java.
4. **Spring Boot Version:** Select the latest stable version (e.g., 3.3.x).
5. **Project Metadata:** Fill in the Group (e.g., `com.example`) and Artifact (e.g., `demo`).
6. **Packaging:** Choose Jar.
7. **Java Version:** Choose 17 or 21.
8. **Dependencies:** Click "Add Dependencies" and search for Spring Web.

Click the **Generate** button to download a ZIP file containing your new project. Extract the ZIP file and open it in your IDE.

Chapter 3

Anatomy of a Spring Boot Project

When you open your freshly generated project, you will notice a standard directory structure. Understanding this structure is crucial for navigating your application.

3.1 The File Structure

- `src/main/java`: This is where all your Java source code lives.
- `src/main/resources`: This folder contains static resources (HTML, CSS), templates, and application configuration files.
- `src/test/java`: This directory is reserved for your unit and integration tests.
- `pom.xml`: The Project Object Model file, which tells Maven how to build your project and lists your dependencies.

3.2 The Maven POM File

Let us examine the `pom.xml` file. You will see that it inherits from a parent project called `spring-boot-starter-parent`.

```
1 <parent>
2   <groupId>org.springframework.boot</groupId>
3   <artifactId>spring-boot-starter-parent</artifactId>
4   <version>3.3.0</version>
5   <relativePath/> <!-- lookup parent from repository -->
6 </parent>
7
8 <dependencies>
9   <dependency>
10     <groupId>org.springframework.boot</groupId>
11     <artifactId>spring-boot-starter-web</artifactId>
12   </dependency>
13   <dependency>
14     <groupId>org.springframework.boot</groupId>
15     <artifactId>spring-boot-starter-test</artifactId>
16     <scope>test</scope>
17   </dependency>
18 </dependencies>
```

Listing 3.1: Snippet of `pom.xml`

Notice that the dependencies do not specify version numbers. The `spring-boot-starter-parent` manages the versions of all Spring and third-party dependencies to ensure they are compatible with each other.

3.3 The Application Class

Inside `src/main/java/com/example/demo`, you will find the main entry point of your application.

```
1 package com.example.demo;
2
3 import org.springframework.boot.SpringApplication;
4 import org.springframework.boot.autoconfigure.SpringBootApplication;
5
6 @SpringBootApplication
7 public class DemoApplication {
8     public static void main(String[] args) {
9         SpringApplication.run(DemoApplication.class, args);
10    }
11 }
```

Listing 3.2: DemoApplication.java

The `@SpringBootApplication` annotation is actually a combination of three different annotations:

- `@Configuration`: Tags the class as a source of bean definitions for the application context.
- `@EnableAutoConfiguration`: Tells Spring Boot to start adding beans based on class-path settings, other beans, and various property settings.
- `@ComponentScan`: Tells Spring to look for other components, configurations, and services in the current package and its sub-packages.

The `main` method uses `SpringApplication.run()` to launch the application.

Chapter 4

Dependency Injection and Beans

To truly understand Spring Boot, you must understand the core concepts of the Spring Framework: Inversion of Control (IoC) and Dependency Injection (DI).

4.1 Inversion of Control (IoC)

In traditional Java programming, if Class A needs to use Class B, Class A will instantiate Class B using the `new` keyword. This creates a tight coupling between the two classes.

Inversion of Control means handing over the responsibility of creating and managing objects to a framework (the Spring IoC Container).

4.2 Beans and Stereotype Annotations

In Spring, an object that is managed by the IoC container is called a **Bean**. You tell Spring which classes should be managed by using Annotations.

- `@Component`: A generic annotation for any Spring-managed component.
- `@Service`: Indicates that an annotated class is a Service (contains business logic).
- `@Repository`: Indicates that an annotated class is a Repository (data access layer).
- `@Controller` / `@RestController`: Indicates that the class serves web requests.

4.3 Dependency Injection

When Spring creates a Bean, it injects the dependencies that the Bean requires.

```
1 import org.springframework.stereotype.Service;
2 import org.springframework.stereotype.RestController;
3
4 @Service
5 public class GreetingService {
6     public String getGreeting() {
7         return "Hello from the Service!";
8     }
9 }
10
11 @RestController
12 public class GreetingController {
13
14     // The dependency
15     private final GreetingService greetingService;
```

```
16
17 // Constructor Injection (Recommended)
18 public GreetingController(GreetingService greetingService) {
19     this.greetingService = greetingService;
20 }
21 }
```

Listing 4.1: Dependency Injection Example

Notice that we did not use the `new` keyword to create `GreetingService`. When Spring creates `GreetingController`, it automatically identifies that it needs a `GreetingService` and passes it into the constructor.

Chapter 5

Building a RESTful API

A REST API (Representational State Transfer) allows different software systems to communicate over the web using standard HTTP methods.

5.1 The RestController

To build an API, we use the `@RestController` annotation. This annotation tells Spring that the class will handle HTTP requests and that the responses should be automatically serialized into JSON and written directly to the HTTP response body.

5.2 HTTP Methods and Mappings

We use mapping annotations to connect HTTP verbs to Java methods:

- `@GetMapping`: To fetch data (HTTP GET).
- `@PostMapping`: To create new data (HTTP POST).
- `@PutMapping`: To update existing data (HTTP PUT).
- `@DeleteMapping`: To delete data (HTTP DELETE).

5.3 Handling Input Parameters

Often, you need to accept input from the client.

5.3.1 Path Variables

Used to extract values from the URI itself (e.g., `/users/123`).

5.3.2 Request Parameters

Used to extract query parameters from the URL (e.g., `/users?status=active`).

5.3.3 Request Body

Used for POST and PUT requests to map the JSON payload in the request body to a Java object.

```

1 package com.example.demo.controller;
2
3 import org.springframework.web.bind.annotation.*;
4 import org.springframework.http.ResponseEntity;
5
6 @RestController
7 @RequestMapping("/api/users")
8 public class UserController {
9
10     // 1. GET Request
11     @GetMapping
12     public String getAllUsers() {
13         return "List of all users";
14     }
15
16     // 2. GET Request with Path Variable
17     @GetMapping("/{id}")
18     public String getUserById(@PathVariable("id") Long id) {
19         return "Returning user with ID: " + id;
20     }
21
22     // 3. GET Request with Query Parameter
23     @GetMapping("/search")
24     public String searchUsers(@RequestParam(value = "name", defaultValue =
25 "John") String name) {
26         return "Searching for user: " + name;
27     }
28
29     // 4. POST Request with Request Body
30     @PostMapping
31     public ResponseEntity<String> createUser(@RequestBody String userJson)
32     {
33         // Normally, you map this to a Java object, not a String
34         return ResponseEntity.status(201).body("User created: " + userJson)
35     }
36 }

```

Listing 5.1: Comprehensive REST Controller Example

If you run your application and navigate to <http://localhost:8080/api/users/1> in your browser, you will see the text "Returning user with ID: 1".

Chapter 6

Data Access with Spring Data JPA

Most applications need to store data. Spring Data JPA drastically reduces the amount of boilerplate code required to implement data access layers.

6.1 Setting Up Dependencies

To connect to a database, you need to add JPA and a database driver to your `pom.xml`. For this tutorial, we will use H2, a lightweight in-memory database perfect for learning.

```
1 <dependency>
2     <groupId>org.springframework.boot</groupId>
3     <artifactId>spring-boot-starter-data-jpa</artifactId>
4 </dependency>
5 <dependency>
6     <groupId>com.h2database</groupId>
7     <artifactId>h2</artifactId>
8     <scope>runtime</scope>
9 </dependency>
```

Listing 6.1: Database Dependencies

6.2 Configuring the Database

Open `src/main/resources/application.properties` and add the following configuration to enable the H2 console and configure the data source:

```
1 spring.datasource.url=jdbc:h2:mem:testdb
2 spring.datasource.driverClassName=org.h2.Driver
3 spring.datasource.username=sa
4 spring.datasource.password=password
5
6 spring.jpa.database-platform=org.hibernate.dialect.H2Dialect
7 spring.jpa.hibernate.ddl-auto=update
8
9 # Enable H2 web console
10 spring.h2.console.enabled=true
11 spring.h2.console.path=/h2-console
```

Listing 6.2: application.properties

6.3 Creating an Entity

An Entity is a Plain Old Java Object (POJO) mapped to a database table.

```

1 package com.example.demo.model;
2
3 import jakarta.persistence.*;
4
5 @Entity
6 @Table(name = "products")
7 public class Product {
8
9     @Id
10    @GeneratedValue(strategy = GenerationType.IDENTITY)
11    private Long id;
12
13    @Column(nullable = false)
14    private String name;
15
16    private double price;
17
18    // Constructors
19    public Product() {}
20
21    public Product(String name, double price) {
22        this.name = name;
23        this.price = price;
24    }
25
26    // Getters and Setters (Omitted for brevity)
27    public Long getId() { return id; }
28    public void setId(Long id) { this.id = id; }
29    public String getName() { return name; }
30    public void setName(String name) { this.name = name; }
31    public double getPrice() { return price; }
32    public void setPrice(double price) { this.price = price; }
33 }

```

Listing 6.3: Product.java

6.4 Creating a Repository

Here is where the magic happens. By extending `JpaRepository`, Spring Boot automatically generates the implementations for all basic CRUD (Create, Read, Update, Delete) operations.

```

1 package com.example.demo.repository;
2
3 import com.example.demo.model.Product;
4 import org.springframework.data.jpa.repository.JpaRepository;
5 import org.springframework.stereotype.Repository;
6
7 import java.util.List;
8
9 @Repository
10 public interface ProductRepository extends JpaRepository<Product, Long> {
11
12    // Spring Data JPA automatically parses the method name
13    // and generates the SQL query!
14    List<Product> findByNameContaining(String keyword);

```

Listing 6.4: ProductRepository.java

You don't need to write a single line of SQL to save, delete, or find products.

Chapter 7

Architectural Best Practices

A well-structured Spring Boot application strictly separates its logic into different layers.

7.1 The Layered Architecture

- **Controller Layer:** Handles HTTP requests and responses. It should contain zero business logic.
- **Service Layer:** Contains the core business rules and logic of the application.
- **Repository Layer:** Responsible for communicating with the database.

7.2 Data Transfer Objects (DTOs)

You should avoid returning your database Entities directly from your REST Controllers. Instead, use DTOs to define the exact shape of the data that the API should receive or return. This prevents accidental exposure of sensitive database fields (like passwords) and decouples the API contract from the database schema.

```
1 package com.example.demo.service;
2
3 import com.example.demo.model.Product;
4 import com.example.demo.repository.ProductRepository;
5 import org.springframework.stereotype.Service;
6 import java.util.List;
7
8 @Service
9 public class ProductService {
10
11     private final ProductRepository productRepository;
12
13     public ProductService(ProductRepository productRepository) {
14         this.productRepository = productRepository;
15     }
16
17     public Product saveProduct(Product product) {
18         // Business logic can be added here
19         return productRepository.save(product);
20     }
21
22     public List<Product> getAllProducts() {
23         return productRepository.findAll();
24     }
25 }
```

Listing 7.1: ProductService.java

Chapter 8

Exception Handling

In a production REST API, you must handle errors gracefully and return meaningful HTTP status codes and JSON error responses, rather than raw stack traces.

8.1 Custom Exceptions

First, define a custom runtime exception for situations where a resource is not found.

```
1 package com.example.demo.exception;
2
3 import org.springframework.http.HttpStatus;
4 import org.springframework.web.bind.annotation.ResponseStatus;
5
6 @ResponseStatus(value = HttpStatus.NOT_FOUND)
7 public class ResourceNotFoundException extends RuntimeException {
8
9     public ResourceNotFoundException(String message) {
10         super(message);
11     }
12 }
```

Listing 8.1: ResourceNotFoundException.java

8.2 Global Exception Handler (@ControllerAdvice)

To catch exceptions globally across all controllers, use @ControllerAdvice.

```
1 package com.example.demo.exception;
2
3 import org.springframework.http.HttpStatus;
4 import org.springframework.http.ResponseEntity;
5 import org.springframework.web.bind.annotation.ControllerAdvice;
6 import org.springframework.web.bind.annotation.ExceptionHandler;
7
8 import java.time.LocalDateTime;
9 import java.util.HashMap;
10 import java.util.Map;
11
12 @ControllerAdvice
13 public class GlobalExceptionHandler {
14
15     @ExceptionHandler(ResourceNotFoundException.class)
16     public ResponseEntity<Object> handleNotFound(ResourceNotFoundException
17 ex) {
18         Map<String, Object> body = new HashMap<>();
19         body.put("timestamp", LocalDateTime.now());
20     }
21 }
```

```
19     body.put("message", ex.getMessage());
20     body.put("status", HttpStatus.NOT_FOUND.value());
21
22     return new ResponseEntity<>(body, HttpStatus.NOT_FOUND);
23 }
24 }
```

Listing 8.2: GlobalExceptionHandler.java

Now, if you throw a `ResourceNotFoundException` from your Service, the user will receive a neatly formatted JSON error response.

Chapter 9

Application Configuration and Profiles

Spring Boot makes it easy to externalize your configuration so that you can work with the same application code in different environments.

9.1 The @Value Annotation

You can inject values directly from your `application.properties` file into your Java code.

```
1 import org.springframework.beans.factory.annotation.Value;
2 import org.springframework.web.bind.annotation.RestController;
3
4 @RestController
5 public class ConfigController {
6
7     @Value("${spring.application.name:DefaultApp}")
8     private String appName;
9
10    @Value("${server.port:8080}")
11    private String port;
12 }
```

Listing 9.1: Injecting Properties

9.2 Spring Profiles

In a real-world scenario, you have different databases for development, testing, and production. Spring Profiles allow you to group configuration settings and activate them dynamically.

Create separate property files:

- `application-dev.properties` (Development Database)
- `application-prod.properties` (Production AWS Database)

In your main `application.properties`, you can set the active profile:

```
1 spring.profiles.active=dev
```

Listing 9.2: Setting Active Profile

Chapter 10

Testing Spring Boot Applications

Testing is a first-class citizen in Spring Boot. The `spring-boot-starter-test` dependency includes JUnit 5, Mockito, and Spring Test utilities.

10.1 Unit Testing

Unit tests focus on testing a single class in isolation. You can use Mockito to “mock” (fake) the dependencies.

```
1 package com.example.demo.service;
2
3 import com.example.demo.model.Product;
4 import com.example.demo.repository.ProductRepository;
5 import org.junit.jupiter.api.Test;
6 import org.junit.jupiter.api.extension.ExtendWith;
7 import org.mockito.InjectMocks;
8 import org.mockito.Mock;
9 import org.mockito.Mockito;
10 import org.mockito.junit.jupiter.MockitoExtension;
11 import java.util.Optional;
12 import static org.junit.jupiter.api.Assertions.assertEquals;
13
14 @ExtendWith(MockitoExtension.class)
15 public class ProductServiceTest {
16
17     @Mock
18     private ProductRepository productRepository;
19
20     @InjectMocks
21     private ProductService productService;
22
23     @Test
24     public void testFindProduct() {
25         Product mockProduct = new Product("Laptop", 1200.0);
26
27         // Define the mock behavior
28         Mockito.when(productRepository.findById(1L))
29             .thenReturn(Optional.of(mockProduct));
30
31         // Call the service
32         Optional<Product> result = productService.findById(1L);
33
34         // Verify the result
35         assertEquals("Laptop", result.get().getName());
36     }
37 }
```

Listing 10.1: ProductServiceTest.java

Chapter 11

Production-Ready Features

When your application moves to production, you need to monitor its health. Spring Boot Actuator provides endpoints to monitor and interact with your application.

11.1 Adding Actuator

Add the dependency to your `pom.xml`:

```
1 <dependency>
2   <groupId>org.springframework.boot</groupId>
3   <artifactId>spring-boot-starter-actuator</artifactId>
4 </dependency>
```

By default, Actuator only exposes the `/health` and `/info` endpoints over the web for security reasons. You can expose more endpoints by adding this to your properties:

```
1 management.endpoints.web.exposure.include=*
```

Listing 11.1: Exposing Actuator Endpoints

Navigating to `http://localhost:8080/actuator/health` will return the health status of your application and database connections.

Chapter 12

Packaging and Deployment

One of Spring Boot's greatest strengths is its deployment model.

12.1 The Executable JAR

Spring Boot packages your application, along with the embedded Tomcat server and all dependencies, into a single, fat JAR file.

To build this JAR, open your terminal and run:

```
1 # For Maven
2 ./mvnw clean package
3
4 # For Gradle
5 ./gradlew build
```

12.2 Running the Application

Once the build is complete, you will find a `.jar` file in the `target` directory (Maven). You can run this file on any machine that has Java installed, without needing to configure a web server!

```
1 java -jar target/demo-0.0.1-SNAPSHOT.jar
```

12.3 Next Steps in Your Journey

Congratulations! You now understand the fundamental concepts required to build an application with Spring Boot. To master enterprise Java, consider exploring the following advanced topics next:

- **Spring Security:** For Authentication (JWT) and Authorization (Role-Based Access Control).
- **Spring Cloud:** For building distributed Microservices architectures, Service Discovery, and API Gateways.
- **Docker and Kubernetes:** For containerizing your fat JAR and orchestrating deployments in the cloud.